Smart City Interop Platform — Description

Last updated: 10 December 2025

This document consolidates every detail of the SOC (Smart Operations Center) demonstration stack: services, contracts, endpoints, data stores, orchestration, gateway policies, client behavior, Docker topology, runbooks and test commands. Use it as the single source of truth when handing the project over for submission.

---

1. Vision & Scope

- Showcase **interop across REST, SOAP, GraphQL and gRPC** behind a single gateway (Kong) secured via JWT + API key.
- Provide a **SmartFlow orchestrator** that consolidates mobility, air quality and emergency data, including SSE feeds.
- Deliver a **React dashboard** that exposes AQI dashboards, multimodal trip planning, municipal directory/tickets and live alerts.
- Keep all data in **PostgreSQL** (mobility, directory, emergency) with migrations, and **Redis** for shared caching.
- Offer a **full Docker Compose environment** (databases, services, orchestrator, gateway, frontend) ready to `docker compose up`.

---

2. Runtime Inventory

Service / Component	Protocol	Container	Port(s)	Data / Dependencies
Mobility REST	HTTP/JSON	`mobility-rest`	8080	PostgreSQL `pg_mobility_data` (table `booking`).
AirQuality SOAP	SOAP 1.1	`airquality-soap`	8081	In-memory dataset (Grand Tunis).
Emergency gRPC	gRPC + HTTP actuator	`emergency-service`	50051 / 9090	PostgreSQL `pg_emergency_data`; synthetic alert simulator (default 5 s).
Directory GraphQL	HTTP/GraphQL	`directory-service`	8082	PostgreSQL `pg_directory_data` (Flyway seeds).
SmartFlow Orchestrator	HTTP/JSON + SSE	`smartflow`	7000	Calls REST/SOAP/gRPC, caches AQI in Redis.

Service / Component	Protocol	Container	Port(s)	Data / Dependencies
Redis cache	RESP	`redis-cache`	6379	Optional; SmartFlow falls back to in-memory TTL store.
Kong Gateway	HTTP/TLS	`kong`	8000 / 8443	DB-less config `gateway/kong/kong.yml`.
React Dashboard	HTTP static	`web-dashboard`	4173	Consumes `/api/*` via Kong.
PostgreSQL (x3)	SQL	`postgres-mobility`, `postgres-directory`, `postgres-emergency`	5432 (internal)	Persist data for each bounded context (mounted volumes `pg_*_data`).

Networks: `backend` (all services + databases) and `frontend` (kong + web dashboard). Docker volumes preserve each Postgres dataset between runs.

---

3. Services in Detail

3.1 Mobility REST (`services/mobility-rest`)

- **Stack**: Spring Boot 3, Spring Data JPA, PostgreSQL.
- **Base path**: `/api` (proxied by Kong under `/api/mobility`).
- **Endpoints** (see `MobilityController` + `contracts/openapi/mobility-openapi.yaml`):
  - `GET /lines?mode=&status=` – list lines filtered by mode (`BUS`, `METRO`, `BIKE`) or status.
  - `GET /lines/{id}` – single line metadata.
  - `GET /lines/{id}/schedule` – generated schedule for a line (intervals per stop).
  - `GET /stops/{stopId}/arrivals` – rolling arrival board with jitter ±1 min.
  - `POST /bookings` – persist a booking (`BookingRequest` with `lineId`, `stopFrom`, `stopTo`, `datetime`, `userId`).
  - `GET /bookings?userId=` – list persisted bookings (filter per citizen).
- **Database**: `schema.sql` executed on startup, persisted via `pg_mobility_data`.
- **Runtime env vars**: `SPRING_DATASOURCE_URL/USERNAME/PASSWORD`, `SPRING_SQL_INIT_MODE`, `SPRING_JPA_DEFER_DATASOURCE_INITIALIZATION`.
- **Local commands**:

```
(cd services/mobility-rest && mvn spring-boot:run)
(cd services/mobility-rest && mvn test)
docker compose up -d --build mobility-rest
```

3.2 AirQuality SOAP (`services/airquality-soap`)

- **Stack**: Java 17 + JAX-WS standalone.

- **WSDL**: `contracts/soap/airquality.wsdl` defines operations `GetAQI`, `GetPollutants`, `CompareZones`.
- **Dataset**: Grand Tunis zones (`CENTRE`, `LAC2`, `LA_MARSA`, `SIDI_BOU_SAID`, `ARIANA`, `BEN_AROUS`).
- **Endpoints** (document/literal SOAP 1.1):
  - `GetAQI(ZoneRequest)` → `AQIReport` (value, category, trend, recommendation).
  - `GetPollutants(ZoneRequest)` → `PollutantResponse` (list of pollutant levels w/ unit µg/m³).
  - `CompareZones(CompareZonesRequest)` → `ComparisonResult` (delta AQI, dominant pollutant).
- **Runtime env vars**: `SOAP_BIND_ADDRESS` (default `http://0.0.0.0:8081/airquality`).
- **Local commands**:

```
(cd services/airquality-soap && mvn package && java -jar target/*.jar)
docker compose up -d --build airquality-soap
```

3.3 Emergency gRPC (`services/emergency-grpc`)

- **Stack**: Spring Boot 3, `grpc-server-spring-boot-starter`, PostgreSQL, Reactor for broadcasting.
- **Proto**: `contracts/grpc/emergency.proto` with message types `Alert`, `AlertAck`, `RecentRequest`, `AlertList`, `AlertFilter` and enum `Severity`.
- **RPCs**:
  - `SendAlert(Alert)` → `AlertAck` (IDs auto-generated `live-` when missing; timestamps default to `Instant.now()`; severity defaults to `INFO`).
  - `GetRecentAlerts(RecentRequest)` → `AlertList` (filters by severity, types, zones; limit capped at 50).
  - `SubscribeAlerts(AlertFilter)` → `stream Alert` (sends snapshot first, then pushes via Reactor sink).
- **Data**: PostgreSQL table `emergency_alert` with Flyway migrations + seeds (six demo alerts). All timestamps stored as `TIMESTAMPTZ`.
- **Synthetic alert producer**: `SyntheticAlertScheduler` (config via `EMERGENCY_SIMULATOR_*`). Generates `sim-xxxxxxxxxx` alerts every `period-ms` (default 5000) with random type/zone/severity/message. Disable by `EMERGENCY_SIMULATOR_ENABLED=false` during automated tests.
- **Runtime env vars**: `EMERGENCY_DB_*`, `GRPC_PORT`, `MANAGEMENT_HTTP_PORT`, `EMERGENCY_SIMULATOR_*`.
- **Local commands**:

```
(cd services/emergency-grpc && mvn spring-boot:run)
(cd services/emergency-grpc && mvn test)
docker compose up -d --build emergency-service
```

- **Inspection**: `docker compose exec postgres-emergency psql -U emergency -c "select * from emergency_alert order by event_timestamp desc limit 10;"`.

3.4 Directory GraphQL (`services/directory-graphql`)

- **Stack**: Spring Boot 3, Spring GraphQL servlet stack, PostgreSQL.

- **Schema**: `contracts/graphql/directory-schema.graphql` (queries `service`, `services`, `tickets`; mutations `createTicket`, `closeTicket`).
- **Runtime**: loads `.graphql` files from `classpath:/graphql`, serves on `POST /graphql` (JSON body `{ "query": "{ services { id } }" }`).
- **Persistence**: Flyway migrations create `directory_service`, `service_hours`, `directory_ticket`; seeds include six demo services + tickets.
- **Env vars**: `PORT`, `DIRECTORY_DB_URL`, `DIRECTORY_DB_USERNAME`, `DIRECTORY_DB_PASSWORD` (and optional schema override).
- **Local commands**:

```
(cd services/directory-graphql && mvn spring-boot:run)
(cd services/directory-graphql && mvn test)
docker compose up -d --build directory-service
```

---

4. SmartFlow Orchestrator (`orchestrator/smartflow`)

- **Stack**: Node 20, Express, TypeScript, `@grpc/grpc-js`, Axios, `fast-xml-parser`, `p-retry`, Redis client.
- **Routes** (served under `/api/smartflow/*` via Kong):
  - `GET /metrics/overview` → Aggregates `fetchAQI`, `fetchLines`, `fetchArrivals`, `fetchRecentAlerts`; returns status + impacted zones.
  - `POST /trip-plan` → Accepts `{ zoneCode, preferredMode, stopId? }`, calls SOAP AQI + mobility REST to recommend best mode and returns itinerary plus ability to auto-book.
  - `GET /alerts/recent` → Proxy to Emergency `GetRecentAlerts`, supports query params `limit`, `minimumSeverity`, `types`, `zones`.
  - `GET /alerts/stream` → SSE: sends heartbeat every 15 s, pushes snapshots every 5 s (`event: snapshot`) plus live events for each gRPC alert; cleans up timers on disconnect.
- **Clients**:
  - Mobility REST via Axios with 2 s timeout, `AbortController` cancellation.
  - AirQuality SOAP via raw HTTP POST + XML parser, cached in Redis (TTL configurable via `CACHE_DEFAULT_TTL`, fallback to in-memory `Map`).
  - Emergency gRPC via generated client from `contracts/grpc/emergency.proto`.
- **Env vars**: `PORT`, `MOBILITY_BASE_URL`, `AIRQUALITY_SOAP_URL`, `EMERGENCY_GRPC_URL`, `REDIS_URL`, `CACHE_DEFAULT_TTL`.
- **Commands**:

```
(cd orchestrator/smartflow && npm install && npm run dev)
(cd orchestrator/smartflow && npm test)
docker compose up -d --build smartflow
```

- **Logging**: `pino-http` attaches `req.id` to each log; errors bubbled as `{ error: 'orchestration_failed', message: ... }`.

---

5. Gateway Kong (`gateway/`)

- **Mode**: DB-less (declarative `gateway/kong/kong.yml`).
- **Routes**:
  - `/api/mobility/*` → `mobility-rest:8080`.
  - `/api/airquality` → `airquality-soap:8081/airquality`.
  - `/api/directory` → `directory-service:8082/graphql` (POST only).
  - `/api/smartflow/*` → `smartflow:7000` (REST + SSE).
- **Security**:
  - Global plugins: `jwt` (HS256, `iss` must match consumer, exp ≤ 3600 s) + `key-auth` (header `x-api-key`).
  - Consumers: `citizen-app` (`citizen-api-key`, secret `supersecretcitizen`) and `operator-app` (`operator-api-key`, secret `supersecretoperator`).
  - Sample JWT (PowerShell):

```
$payload = '{"iss":"citizen-app","exp":' + ([int][double]::Parse((Get-Date -UFormat %s)) + 900) + '}'
$secret = 'supersecretcitizen'
... # see docs/phase4/gateway-security.md for full function
```

- **TLS**: optional port `8443` using `gateway/certs/server.crt|server.key`. Generate self-signed cert via `openssl req ...`.

---

6. Web Dashboard (`client/web-dashboard`)

- **Stack**: React + Vite + TypeScript, TanStack Query, Context-based auth (API key + JWT stored in `localStorage`).
- **Pages**:
  - **Dashboard** – `GET /api/smartflow/metrics/overview`; shows AQI focus, pollutants cards, alert digest for Grand Tunis.
  - **Planificateur** – `POST /api/smartflow/trip-plan`; uses citizen ID-specific defaults.
  - **Annuaire & Tickets** – GraphQL queries/mutations via `/api/directory`; forms restricted to logged-in citizen.
  - **Alertes** – SSE via `/api/smartflow/alerts/stream`; server snapshots every 5 s, filters for severity/zones/types, manual refresh button hits `/alerts/recent`.
- **Scripts**:

```
cp .env.example .env   # set VITE_GATEWAY_URL, JWT, API key
npm install
npm run dev -- --host   # http://localhost:5173
npm run build           # production bundle
RUN_E2E=1 PLAYWRIGHT_BASE_URL=http://localhost:4173 npm run test:e2e
```

- **Auth UX**: landing card with login/registration toggles; selecting "Accès démo" auto-injects `citizen-api-key` + demo JWT.

---

7. Contracts & Documentation

- `contracts/openapi/mobility-openapi.yaml` — canonical REST schema (Swagger UI available at `/mobility/swagger-ui.html`).
- `contracts/soap/airquality.wsdl` — SOAP contract for code generation/testing.
- `contracts/graphql/directory-schema.graphql` — SDL imported by Spring GraphQL.
- `contracts/grpc/emergency.proto` — used by emergency service and SmartFlow.
- `docs/phase*/` — step-by-step documentation (vision, architecture, SmartFlow, security, frontend, ops, backlog).
- `FullReadme.md` — historic comprehensive readme; superseded by this Description but retained for traceability.

---

8. Docker & Infrastructure

- **Compose file**: `docker-compose.yml` orchestrates all services with proper dependencies and environment variables.
- **Volumes**: `pg_mobility_data`, `pg_directory_data`, `pg_emergency_data` preserve database files between runs.
- **Networks**: `backend` (internal) and `frontend` (exposes Kong + dashboard).
- **Selective rebuild**: `docker compose up -d --build <service>` (e.g., `web-dashboard`, `smartflow`, `emergency-service`).
- **Logs**: `docker compose logs -f <service>`; `smartflow` uses JSON logs, Spring services use standard logging.
- **Health checks**: SmartFlow `/health`, Spring Actuator (emergency service) exposes `/actuator/health` via port 9090.

---

9. Security Summary

- **Mandatory headers** for any call through Kong: `Authorization: Bearer <jwt>` (HS256) and `x-api-key: citizen-api-key`.
- **Token issuance**: use `docs/phase4/gateway-security.md` PowerShell snippet or any HS256 generator (payload requires `iss` set to `citizen-app` or `operator-app`, `exp` ≤ 3600 s from now).
- **TLS** optional but supported; configure cert/key in `gateway/certs` and restart Kong.
- **Session propagation**: SmartFlow and Directory rely on `citizenId` included in request bodies; the dashboard stores it in session context to avoid cross-user leakage.

---

10. Runbook & Verification

1. **Bootstrap**

```
docker compose build
docker compose up -d
docker compose ps
```

Wait for logs `Started MobilityApplication`, `Started DirectoryServiceApplication`, `Started EmergencyServiceApplication`, `SmartFlow orchestrator listening on port 7000`.

2. **Generate a JWT** (PowerShell or Node) and export headers for testing:

```
$env:API_KEY = 'citizen-api-key'
$env:JWT = '<generated-token>'
```

3. **Smoke tests**:
   - Mobility lines: `curl -H "x-api-key: $env:API_KEY" -H "Authorization: Bearer $env:JWT" http://localhost:8000/api/mobility/api/lines`.
   - GraphQL: `curl http://localhost:8000/api/directory -H "Content-Type: application/json" -H ... -d '{"query":"{ services { id name } }"}'`.
   - SmartFlow metrics: `curl http://localhost:8000/api/smartflow/metrics/overview -H ...`.
   - Alerts SSE: `curl -N http://localhost:8000/api/smartflow/alerts/stream -H ...` (observe `event: snapshot` + `data:` lines every ~5 s due to simulator).
4. **Database checks**:
   - Mobility: `docker compose exec postgres-mobility psql -U mobility -c 'select count(*) from booking;'`.
   - Directory: `docker compose exec postgres-directory psql -U directory -c 'select name, category from directory_service;'`.
   - Emergency: `docker compose exec postgres-emergency psql -U emergency -c 'select id, zone, severity from emergency_alert order by event_timestamp desc limit 10;'`.
5. **Restart a single service**: `docker compose up -d --build emergency-service` (useful after code changes in `services/emergency-grpc`).
6. **Run tests prior to submission**:

```
(cd orchestrator/smartflow && npm test)
(cd services/mobility-rest && mvn test)
(cd services/directory-graphql && mvn test)
(cd services/emergency-grpc && mvn test)
(cd client/web-dashboard && npm run build)
RUN_E2E=1 PLAYWRIGHT_BASE_URL=http://localhost:4173 (cd client/web-dashboard && npm run test:e2e)
```

Note: Playwright installs browsers on first run via `npx playwright install`.

7. **Maintenance tips**:
   - Flush Redis cache (if redis is unhealthy): `docker compose exec redis-cache redis-cli FLUSHALL`.

- Disable simulator temporarily: set `EMERGENCY_SIMULATOR_ENABLED=false` in `docker-compose.yml` override or environment, then `docker compose up -d emergency-service`.
- Update TLS certs: place new files under `gateway/certs/` and restart Kong.

---

11. Submission Checklist

- ☑ No leftover binaries (`directory-service-app.jar`, `booking.json` removed).
- ☑ READMEs present for every major service (`services/README.md`, `services/*/README.md`, orchestrator, client).
- ☑ `Description.md` (this file) and `FullReadme.md` kept in sync; README references Description as the authoritative doc.
- ☑ Docker Compose builds cleanly; `docker compose up -d` succeeds on a clean workstation.
- ☑ Synthetic alert simulator running (unless disabled) so the alerts page always shows fresh data.
- ☑ Tests/build commands documented.

This Description captures the entire project from inception (protocol diversity) to the final December 2025 refinements (GraphQL servlet migration, timestamp fixes, SmartFlow streaming snapshots, synthetic alert generator). Use it for onboarding, audits and as the canonical artifact for the submission package.